

TEORÍA DE ALGORITMOS
(75.29) CURSO BUCHWALD - GENENDER

Trabajo Práctico 3

Problemas NP-Completo



15 de junio de 2026

Matias Urbani
113739

Paula Romano
112609

Juan Contino
113861



Índice

1. Introducción	3
2. Demostración NP	3
3. Demostración NP-Completo	4
3.1. Definición de la Reducción	4
3.2. Complejidad de la Reducción	4
3.3. Correctitud de la Reducción	4
4. Backtracking	5
4.1. Mediciones	6
5. Programación Lineal	6
6. Aproximación por Programación Lineal	7
6.1. Complejidad Temporal	8
6.2. Calidad de la aproximación	8
7. Aproximación por algoritmo Greedy	9
7.1. Complejidad Temporal	9

1. Introducción

Scaloni ya está armando la lista de 43 jugadores que van a ir al mundial 2026. Hay mucha presión por parte de la prensa para bajar línea de cuál debería ser el 11 inicial. Lo de siempre. Algunos medios quieren que juegue Roncaglia, otros quieren que juegue Mateo Messi, y así. Cada medio tiene un subconjunto de jugadores que quiere que jueguen. A Scaloni esto no le importa, no va a dejar que la prensa lo condicione, pero tiene jugadores jóvenes a los que esto puede afectarles.

Justo hay un partido amistoso contra Burkina Faso la semana que viene. Oportunidad ideal para poner un equipo que contente a todos, baje la presión y poder aislar al equipo.

El problema es, ¿cómo elegir el conjunto de jugadores que jueguen ese partido (entre titulares y suplentes que vayan a entrar)? Además, Scaloni quiere poder usar ese partido para probar cosas aparte. No puede gastar el amistoso para contentar a un periodista mufa que habla mal de Messi, por ejemplo. Quiere definir el conjunto más pequeño de jugadores necesarios para contentarlos y poder seguir con la suya. Con elegir un jugador que contente a cada periodista/medio, le es suficiente.

Ante este problema, Bilardo se sentó con Scaloni para explicarle que en realidad este es un problema conocido (viejo zorro como es, ya se comió todas las operetas de prensa así que se conoce este problema de memoria). Se sirvió una copa de Gatorei y le comentó:

“Esto no es más que un caso particular del Hitting-Set Problem. El cual es: Dado un conjunto A de n elementos y m subconjuntos $B_1, B_2, \dots, B_m$ de A ($B_i \in A \forall i$), queremos el subconjunto $C \in A$ de menor tamaño tal que C tenga al menos un elemento de cada B_i (es decir, $C \cap B_i \neq \emptyset$). En nuestro caso, son los jugadores convocados, los B_i son los deseos de la prensa, y C es el conjunto de jugadores que deberían jugar contra Burkina Faso si o si”.

Bueno, ahora con un poco más claridad en el tema, Scaloni necesita de nuestra ayuda para ver si obtener este subconjunto se puede hacer de forma eficiente (polinomial) o, si no queda otra, con qué alternativas contamos.

2. Demostración NP

Para demostrar que un problema se encuentra en NP, hay que demostrar que su versión de decisión se puede verificar en tiempo Polinomial.

El problema de decisión del Hitting-Set es. Sea A un conjunto de elementos, $B_1, B_2, \dots, B_m$ Subconjuntos pertenecientes a A , dado un numero k , existe un Hitting-Set de a lo sumo k elementos.

Se puede plantear el siguiente verificador

```
def verificador_hitting_set(A, B, k, C):  
    if len(C) > k:  
        return False  
    if not C.issubset(A):  
        return False  
    for subconjunto in B:  
        interseccion_encontrada = False  
        for jugador in subconjunto:  
            if jugador in C:  
                interseccion_encontrada = True  
                break  
        if not interseccion_encontrada:  
            return False  
    return True
```

Sea n la cantidad de elementos de A y m la cantidad de subconjuntos de B

La complejidad de revisar si C es un subconjunto de A es de $O(n)$ por tener que revisar en el peor caso todos los elementos de A , y la complejidad de los for es $O(nm)$, pues en el peor caso revisaría todos los elementos de cada subconjunto, que podrían ser n elementos en cada uno de los m subconjuntos

La complejidad del verificador es $T(n, m) = O(1) + O(n) + O(nm) = O(nm)$

3. Demostración NP-Completo

Para demostrar que un problema está en NP-Completo se deben demostrar 2 cosas, que el problema está en NP, y que está en NP-Difícil.

Ya se demostró que el problema está en NP, quedaría demostrar que el problema está en NP-Difícil

Para demostrar que un problema está en NP-Difícil, debemos demostrar que un problema que ya sabemos que está en NP-Completo se puede reducir a este, para este caso elegimos Vertex-Cover, es decir, buscamos reducir polinomialmente Vertex-Cover a Hitting-Set

3.1. Definición de la Reducción

El problema de decisión del Vertex-Cover es el siguiente, Sea G un grafo de V vértices y E aristas y un número K , existe un conjunto de a lo sumo K vértices de G , tal que para toda arista de G al menos uno de sus extremos se encuentre en el conjunto. Este problema es NP-Completo

Se puede plantear la siguiente reducción, llamo a la caja negra que resuelve Hitting-Set pasándole los vértices del grafo como valores de A , por cada arista planteo un conjunto que incluya sus dos vértices como subconjuntos pertenecientes a B , y le paso el mismo valor de K .

3.2. Complejidad de la Reducción

Esta transformación recorre los vértices y las aristas del grafo una única vez, por lo que su tiempo de ejecución es $O(V + E)$. Al ser en tiempo lineal la reducción es polinomial.

3.3. Correctitud de la Reducción

Demuestro que siempre que existe un Vertex-Cover de tamaño a lo sumo k en el grafo original, existe un Hitting-Set de tamaño a lo sumo K en los conjuntos A y B creados.

Si existe un Vertex-Cover de tamaño a lo sumo k en un grafo, necesito a lo sumo k vértices tal que todas las aristas tengan al menos uno de sus vértices en el Vertex-Cover. al pedirle al Hitting-Set que plantee un universo donde los vértices son los elementos de A y cada arista, un Conjunto perteneciente a B , este deberá encontrar un vértice por, siempre podrá encontrar esos mismos vértices, que son a lo sumo K , pertenecen al universo A , y cubren todos los subconjuntos de B .

Demuestro que siempre que existe un Hitting-Set de a lo sumo K elementos después de aplicar la transformación, existe un Vertex-Cover en el grafo original.

Si existe un Hitting-Set de tamaño a lo sumo k en los conjuntos creados, significa que todos los conjuntos de B se pueden cubrir con a lo sumo k elementos de A . Si se eligieran esos mismos elementos en el grafo original, tendríamos un conjunto de a lo sumo k vértices del grafo original tal que todas las aristas están cubiertas por estos (porque cubren todos los conjuntos de B que representan esas aristas), lo que obtiene siempre un Vertex-Cover de tamaño a lo sumo K .

Conclusión: De esta forma, al reducir un problema NP-Completo conocido a Hitting Set y habiendo probado anteriormente que pertenece a NP, concluimos que el problema Hitting-Set es NP-Completo.

4. Backtracking

Una solución del algoritmo por Backtracking seria la siguiente

```
def primer_no_cubierto(actual, conjuntos):

    for b in conjuntos:
        if len(actual & b) == 0:
            return b
    return None

def backtrack(actual, conjuntos, mejor):
    # explora recursivamente, guardando la mejor solucion

    # poda: si ya superamos o igualamos la mejor solucion, cortar
    if len(actual) >= mejor["tam"]:
        return

    b = primer_no_cubierto(actual, conjuntos)

    if b is None:

        mejor["tam"] = len(actual)
        mejor["solucion"] = set(actual)
        return

    for elem in b:
        if elem in actual:
            continue
        backtrack(actual | {elem}, conjuntos, mejor)

def hitting_set_backtracking(n, conjuntos):
    # cota superior inicial: n+1
    mejor = {"tam": n + 1, "solucion": None}

    backtrack(frozenset(), conjuntos, mejor)

    return mejor["tam"], mejor["solucion"]

def main():
    if len(sys.argv) < 2:
        print("Uso: python3 tp3.py ruta/a/listado.txt")
        sys.exit(1)

    path = sys.argv[1]
    n, m, conjuntos, nombres = leer_instancia(path)

    inicio = time.perf_counter()
    tam, solucion = hitting_set_backtracking(n, conjuntos)
    fin = time.perf_counter()

    jugadores_solucion = sorted(nombres[i] for i in solucion)
```

```
print(f"n = {n}, m = {m}")
print(f"Tamaño del Hitting-Set optimo: {tam}")
print(f"Hitting-Set: {jugadores_solucion}")
print(f"Tiempo de ejecucion: {fin - inicio:.6f} segundos")
```

4.1. Mediciones

Para resolver Hitting set implementamos un algoritmo de Backtracking que podando ramas cuando la solución parcial ya supera el tamaño óptimo encontrado hasta el momento. La idea es tomar el primer subconjunto B_i no cubierto y ramificar eligiendo cada uno de sus elementos como candidato a incorporar al Hitting-set, para que en cada rama ese subconjunto quede cubierto.

Sets de datos y Correctitud:

5.txt :

n = 10, m = 5

Óptimo: 2 — Resultado: 2

Hitting-Set: Barcon't, Messi

Tiempo: 0.000029s

10pocos.txt:

n = 17, m = 10

Óptimo: 3 — Resultado: 3

Hitting-Set: Casco, Chiquito Romero, Di Maria

Tiempo: 0.000114s

10todos.txt:

n = 33, m = 10

Óptimo: 10 — Resultado: 10

Hitting-Set: Almada, Correa, Cuti, Dibu, Guido Rodriguez,

Lautaro, Martinez Quarta, Messi, Montiel, Palacios

Tiempo: 0.048090s

50.txt:

n = 31, m = 50

Óptimo: 6 — Resultado: 6

Hitting-Set: Armani, Barcon't, Casco, Dybala, Langoni, Tucu Pereyra

Tiempo: 0.034634s

100.txt:

n = 31, m = 100

Óptimo: 9 — Resultado: 9

Hitting-Set: Barcon't, Buonanotte, Dibu, El fantasma de la B, Gallardo, Langoni, Palermo, Soule, Wachoffisde Abila

Tiempo: 4.054723s

5. Programación Lineal

Para resolver este problema por Programación Lineal, planteamos el siguiente Modelo.

Definimos n variables binarias $A_1, A_2, \dots, A_n$ que representan si cada uno de los n elementos de A están en la solución óptima o no (Toman valor 1 cuando el elemento está en la solución óptima y 0 cuando no está).

Definimos una restricción para cada elemento de B_i de B de la forma

$$\sum_{j \in B_i} A_j \geq 1 \quad (1)$$

y buscamos la solución que minimice

$$\sum_{i \in A} A_i \quad (2)$$

Este sería el código de la solución

```
def resolver_pl(n, conjuntos, nombres):

    modelo = LpProblem("SetCover", LpMinimize)

    # Variables binarias
    x = LpVariable.dicts("x", range(n), cat="Binary")

    # Función objetivo: minimizar cantidad de elementos elegidos
    modelo += lpSum(x[i] for i in range(n))

    # Restricciones: cada conjunto debe tener al menos un elemento elegido
    for i, conjunto in enumerate(conjuntos):
        modelo += lpSum(x[j] for j in conjunto) >= 1

    # Resolver
    modelo.solve(PULP_CBC_CMD(msg=False))

    solucion = [
        nombres[i]
        for i in range(n)
        if value(x[i]) > 0.5
    ]

    return solucion, value(modelo.objective)
```

6. Aproximación por Programación Lineal

La aproximación por Programación Lineal Continua propuesta es la siguiente

```
def resolver_pl_relajado(n, conjuntos, nombres):

    modelo = LpProblem("HittingSetRelajado", LpMinimize)

    # Variables reales entre 0 y 1
    x = LpVariable.dicts(
        "x",
        range(n),
        lowBound=0,
        upBound=1,
        cat="Continuous"
```

```

)

# Función objetivo
modelo += lpSum(x[i] for i in range(n))

# Restricciones: cada conjunto debe estar cubierto
for conjunto in conjuntos:
    modelo += lpSum(x[j] for j in conjunto) >= 1

# Resolver relajación lineal
modelo.solve(PULP_CBC_CMD(msg=False))

# b = tamaño del conjunto más grande
b = max(len(conjunto) for conjunto in conjuntos)

umbral = 1 / b

# Redondeo
solucion = [
    nombres[i]
    for i in range(n)
    if value(x[i]) >= umbral
]

# Valores fraccionales (opcional, para análisis)
valores_relajados = {
    nombres[i]: value(x[i])
    for i in range(n)
}

return solucion, valores_relajados, b, umbral

```

6.1. Complejidad Temporal

La complejidad temporal del algoritmo sería de $O(nm)$, que es el costo de crear las restricciones del modelo, que recorre todos los elementos de cada conjunto, sin contar el costo de resolver el modelo por programación lineal continua, el cual es de $O(n^9)$, siendo n la cantidad de variables y restricciones del modelo.

Contando la complejidad de resolver el modelo, la complejidad temporal sería en el peor caso $O((n + m)^9)$

6.2. Calidad de la aproximación

La aproximación elige al menos un elemento de cada subconjunto, porque cada subconjunto debe tener al menos una variable con valor mayor que $1/b$ para que la suma de sus valores pueda ser mayor o igual a 1.

La aproximación de un modelo relajado de programación lineal, corresponde al umbral utilizado para redondear los datos, que en este caso es b .

El algoritmo podría elegir a lo sumo b elementos de cada subconjunto, si todos tuvieran el valor $1/b$, por lo que en el peor caso, el algoritmo elige b elementos por cada conjunto cuando podría elegir solo 1, por lo que es una b -aproximación

7. Aproximación por algoritmo Greedy

La solución Greedy propuesta consiste en usar siempre los elementos que cubrirían mas subconjuntos todavía no cubiertos, hasta que todos los subconjuntos este cubiertos.

Este es el código de la solución

```
def resolver_greedy(n, conjuntos, nombres):

    conjuntos_no_cubiertos = set(range(len(conjuntos)))

    solucion = []

    while conjuntos_no_cubiertos:

        frecuencia = defaultdict(int)

        for idx_conjunto in conjuntos_no_cubiertos:
            for jugador in conjuntos[idx_conjunto]:
                frecuencia[jugador] += 1

        mejor_jugador = max(
            frecuencia,
            key=frecuencia.get
        )

        solucion.append(nombres[mejor_jugador])

        cubiertos = {
            i
            for i in conjuntos_no_cubiertos
            if mejor_jugador in conjuntos[i]
        }

        conjuntos_no_cubiertos -= cubiertos

    return solucion
```

7.1. Complejidad Temporal

En cada iteración, el algoritmo recorre todos los jugadores de cada conjunto, por lo que realiza $O(nm)$ operaciones.

En el peor caso, debe incluir 1 jugador de cada conjunto, por lo que realizaría m iteraciones

$$T(n, m) = mO(nm) = O(nm^2)$$